RETAILMART V3 · SQL

Introduction to SQL & Databases

— Siraj Sir

📱 WhatsApp Announcement

WELCOME TO THE SQL MASTERCLASS!

Every app you love — Netflix, Swiggy, PhonePe — is just a pretty shell on top of a massive Database. The UI is the face; the Database is the brain. Today we learn to speak directly to that brain.

Today's Focus (pure concepts — no installation yet):

- Data vs Information vs a Database
- SQL vs NoSQL — why banks choose tables
- Tables, Rows, Columns, Keys & Schemas
- The SQL command families + who uses SQL (careers)

Course Portal: https://starlordali4444.github.io/PostgreSql/
Setup links shared at the end — download before our next class!

Let's build the foundation, brick by brick! — Siraj Sir

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
The World of Databases	35 mins	Data vs Information, What is a DBMS / RDBMS?, Relational vs Non-Relational, Database history & PostgreSQL
Think About It: Databases	15 mins	Data vs Information, SQL vs NoSQL — reasoning out loud
Anatomy of a Table	30 mins	Tables, Rows, Columns, Primary Keys & Foreign Keys, Schemas & Database Hierarchy
Think About It: Table Design	15 mins	Identify keys & columns, Sketch tables for real apps
SQL — The Language	25 mins	What is SQL?, DDL / DML / DQL / DCL / TCL, Declarative vs Imperative, The Query Pipeline
Think About It: SQL Commands	15 mins	Classify DDL / DML / DQL, Declarative thinking
Roles & Career Paths	15 mins	Who uses SQL?, Skills Matrix, SQL vs Python, Setup link for Setup & Onboarding
Think About It + Wrap-up	10 mins	Roles & SQL vs Python, Self-check, setup preview

STORY

Before You Even Reached This Class Today... (1/2)

Let us trace your morning. Every single step triggered a database query somewhere in the world:

```
[Your Morning — Decoded]
6:30 AM  Phone alarm rings
         → Your alarm app READ your settings from a local SQLite database.
         → Query: SELECT alarm_time, ringtone FROM alarms WHERE enabled = true;


6:35 AM  You check WhatsApp (17 unread messages)
         → WhatsApp's server ran a query to FETCH your unread messages.
         → 2 billion messages are INSERTED into databases every single day.


7:00 AM  UPI payment at the chai stall — ₹20 via PhonePe
         → Your bank DEBITED ₹20, chai wala's bank CREDITED ₹20.
         → That is TWO database UPDATEs wrapped in a TRANSACTION.
         → If your phone lost signal mid-payment? The database ROLLS BACK both.
         → Your ₹20 is safe. That is the power of ACID.
```

STORY

Before You Even Reached This Class Today... (2/2)

```
7:15 AM  Scrolled Instagram for 10 minutes
         → Instagram ran ~50 SELECT queries: your feed, stories, reels, likes.
         → 500M+ daily users. All metadata lives in PostgreSQL.


7:30 AM  Booked an Ola/Uber
         → Driver locations UPDATE every 4 seconds in a real-time database.
         → Your ride fare = a query joining distance, surge pricing, and promo tables.


8:00 AM  Opened Swiggy, searched 'Biryani'
         → SELECT * FROM restaurants WHERE menu LIKE '%Biryani%'
         →   AND city = 'your_city' ORDER BY rating DESC;
         → 47 results in 0.3 seconds across 200,000+ restaurants.


TOTAL: ~127 database queries before 9 AM.
       And you did not write a single one.
       Someone did. That someone had the skill you are about to learn.
```

By the end of this course, you will be that someone. You will be able to talk directly to databases — ask questions, get answers, find patterns that no one else can see. That language is called SQL. And it starts today.

REVEAL

WHY THIS MATTERS FOR YOUR CAREER

Every company — from a 5-person startup to Reliance — stores its business data in databases. The people who can QUERY that data, EXTRACT insights, and ANSWER business questions are called Data Analysts. They are the translators between raw data and business decisions.

LinkedIn's 2025 India report: 'SQL' appears in 73% of all Data Analyst job postings. Not Python. Not Excel. Not Power BI. SQL.

In 30 sessions, you go from 'what is a database?' to writing queries that can analyze 2.7 million rows of real retail data — the same kind of queries running at Flipkart, Amazon, and Myntra right now.

DEFINITION

The Data → Information Pipeline

Raw Data:

The number '42' sitting alone in a file. Is it a temperature? An age? A shoe size? A pin code? Nobody knows. Raw data is meaningless without context.

Information:

'The customer is 42 years old.' NOW the number has meaning. Information = Data + Context + Structure. A database's job is to store data in a way that it automatically becomes queryable information.

Let us make this real:

```
[Data vs Information — Everyday Examples]
RAW DATA (meaningless alone)     →   INFORMATION (data + context)
──────────────────────────────────────────────────────────────────

9876543210                        →   Rahul's phone number
₹247                              →   Today's Swiggy order total
4.2                               →   Average rating of Biryani Blues
2025-05-15                        →   Date customer registered
TRUE                              →   Customer has verified email


Same numbers. Add COLUMN NAMES and TABLE STRUCTURE → instant meaning.
```

DEFINITION

Database vs DBMS — The Warehouse and the Manager (1/2)

Database:

An organized collection of data stored electronically. Think of it as a massive digital filing system with millions of records — but unlike paper files, it can be searched in milliseconds.

DBMS (Database Management System):

The intelligent software layer that sits ON TOP of those files. You never touch the raw files on disk. You talk to the DBMS and it handles everything — reading, writing, security, crash recovery, multi-user access.

```
[The Analogy]
Think of a Bank:
```

DEFINITION

Database vs DBMS — The Warehouse and the Manager (2/2)

```
The VAULT (locked room with cash)  =  Database (physical files on disk)
The BANK MANAGER + tellers + rules  =  DBMS (PostgreSQL)
The CUSTOMER (you)                  =  Application / SQL User


You NEVER walk into the vault yourself.
You go to the counter, fill a form (SQL query), and the manager
handles everything — checks your identity, finds your account,
processes the transaction, gives you a receipt.


That is exactly how a DBMS works.
```

RDBMS (Relational DBMS):

A DBMS that stores data in TABLES (rows and columns) and allows RELATIONSHIPS between tables using Keys. 'Relational' does not mean 'related' — it comes from 'relation', a mathematical term for a table. PostgreSQL, MySQL, Oracle, SQL Server — all RDBMS.

STORY

SQL vs NoSQL — The WhatsApp Group vs The Excel Sheet (1/2)

Your college class has a WhatsApp group with 80 students. The class representative posts:

'Everyone send your name, roll number, phone number, and blood group.'

What happens next? Absolute chaos:

- Ravi sends: 'Ravi Kumar, 42, 98765xxxxx, B+'
- Priya sends: 'B+, Priya, my number is 87654xxxxx, roll 15'
- Amit sends a voice note instead
- Someone sends just 'B+' with no name
- 3 people reply with memes
- 10 people do not reply at all

STORY

SQL vs NoSQL — The WhatsApp Group vs The Excel Sheet (2/2)

Now the CR has to submit this data to the college office. She spends 4 HOURS manually extracting 80 students' info from 200+ messages. That is UNSTRUCTURED data. That is NoSQL — flexible, fast to write, but a nightmare to analyze.

Alternative:

The CR creates a Google Sheet with columns: Name | Roll No | Phone | Blood Group. She shares the link. Every student fills THEIR row. Fixed columns, fixed format, no duplicates. Done in 20 minutes. That is STRUCTURED data. That is SQL.

The key insight:

Both stored the same information. But when the principal asks 'How many students are B+ blood group?' — the spreadsheet answers in 2 seconds. The WhatsApp group? You are still scrolling through memes.

Relational (SQL) vs Non-Relational (NoSQL) — Complete Comparison

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data Structure	Strict tables (rows/columns)	Flexible (JSON, key-value)
Schema	Predefined and enforced	Schema-less or dynamic
Relationships	Foreign Keys (explicit links)	Embedded or manual refs
Data Integrity	ACID guaranteed	Eventual consistency
Query Language	SQL (standardized since 1986)	Varies per engine
Scaling	Vertical (bigger server)	Horizontal (more servers)
Best For	Banking, e-commerce, analytics	Chat, caching, IoT, logs

SQL Engines : PostgreSQL, MySQL, Oracle, SQL Server, SQLite
NoSQL Engines: MongoDB (documents), Redis (cache), Cassandra (wide-column),
DynamoDB (key-value), Neo4j (graph), ClickHouse (analytics)

REAL WORLD

The 80/20 Rule of Databases

Approximately 80% of all enterprise data lives in Relational (SQL) databases. The remaining 20% is in NoSQL systems.

Even companies that famously adopted MongoDB (like Uber) migrated their critical financial data BACK to PostgreSQL because they needed ACID guarantees for money transactions.

Modern companies use BOTH — this pattern is called Polyglot Persistence:

- PostgreSQL — Core business data (users, orders, products, transactions)
- Redis — Hot cache + real-time sessions (< 1ms response)
- Elasticsearch — Full-text search (the search bar on e-commerce sites)
- MongoDB / S3 — Event logs, analytics blobs, user-uploaded files

The 'source of truth' — the system that MUST never lose a record — is almost always an RDBMS. That is PostgreSQL's home territory.

The Evolution of Databases — A Quick Timeline

Year	What Happened	Why It Matters
1960s	Data stored in flat files (like .txt, .csv)	No structure, no safety
1970	Edgar Codd publishes the Relational Model at IBM	Tables, rows, columns born
1974	IBM builds System R — first SQL prototype	SQL language is created
1979	Oracle releases first commercial RDBMS	Databases become a business
1986	SQL becomes ANSI standard	One language, all engines
1996	PostgreSQL 6.0 released (open-source, free)	Enterprise power, zero cost
2009	MongoDB popularizes NoSQL	Flexible schemas for web apps
2012	Google's Spanner — distributed SQL at global scale	SQL is NOT dead, it evolved
2024	PostgreSQL 17 — JSON, partitioning, logical replication	Most advanced open-source DB
2025	PostgreSQL 18 — YOU are learning this right now	Cutting-edge, production-ready

Key takeaway: SQL was created in 1974. It is 51 years old.
It has outlived 5 generations of programming languages.
It is not going anywhere. It is the lingua franca of data.

THINK ABOUT IT — The World of Databases

Discuss these in pairs — reason out loud. No trick answers, just thinking.

EASY

P1.1 — Data or Information?

Classify each item as RAW DATA or INFORMATION. If it is raw data, add context to make it information:

Example: '47' on its own is raw data. '47 units left in Mumbai store #12' is information — context gives the number meaning.

a) : ₹1,499
b) : Priya Sharma has 2,340 loyalty points and is a Gold-tier member
c) : TRUE
d) : Mumbai store #47 had ₹12.5 lakhs revenue in April 2025
e) : 2025-05-15
f) : 47

Bonus: For each raw data item, give TWO different contexts that change its meaning completely.

MEDIUM

P1.2 — SQL or NoSQL? Justify Every Choice

For each scenario, pick SQL (PostgreSQL) or NoSQL and explain WHY. Name the specific property that makes your choice correct:

a) : PhonePe transfers ₹5,000 from Account A to Account B. The debit and credit MUST both succeed or both fail. No partial states allowed.

b) : WhatsApp stores 100 billion messages per day. Each message has different attachments — some have images, some have documents, some are just text.

c) : IRCTC manages railway reservations. Seat 23 in Coach S4 cannot be sold to two passengers simultaneously.

d) : A gaming platform stores player session data — each session has wildly different attributes depending on the game.

e) : A hospital records patient allergies, prescriptions, and ongoing treatments. Lives depend on 100% accuracy.

f) : Zomato caches restaurant menus for instant display. Menu changes are okay to show 5 minutes late.

HARD

P1.3 — The Aadhaar Database Challenge

India's Aadhaar system stores biometric and demographic data for 1.4 billion residents. Think about this:

a) : Should Aadhaar use SQL or NoSQL for its CORE identity data? Defend your answer using at least 3 properties (ACID, schema, integrity, etc.)

b) : Aadhaar links to your bank, your phone, your ration card, your gas subsidy. Draw the RELATIONSHIP between these 5 systems. Which system is the 'source of truth'?

c) : What happens if two people accidentally get the same Aadhaar number? What database concept prevents this?

d) : The government asks: 'How many Aadhaar holders in Maharashtra are above 60 years old?' Would this be faster in SQL or NoSQL? Why?

CRAZY

P1.4 — The Great Database Debate

Argue BOTH sides. There is no single right answer — this builds interview thinking:

Debate: : 'NoSQL will replace SQL in 10 years.'

Write 3 arguments FOR (flexible schemas, horizontal scaling, JSON-native apps, cloud-first)
Write 3 arguments AGAINST (ACID is non-negotiable for finance, 50+ years of tooling, SQL is evolving too — PostgreSQL now supports JSON natively)

Then give your personal verdict: Which side wins in the Indian market and why?

STORY

Your Phone Has a Database — You Just Never Noticed

Open the Contacts app on your phone. You probably have 300-500 contacts. Each contact has:

- A name
- A phone number (maybe two — personal and work)
- An email (maybe)
- A company name (maybe)
- A photo (maybe)

Congratulations — your phone is running a database. Specifically, a SQLite database.

When you search for 'Rahul' and 3 contacts appear in 0.1 seconds? That was:

```
[What Your Phone Just Did]
SELECT name, phone_number
FROM contacts
WHERE name LIKE '%Rahul%';


That is an actual SQL query running on your phone right now.
```

Now think: what if your contacts app had no structure? What if names, numbers, and emails were all dumped into one big text file with no columns? Finding 'Rahul' would mean scanning every character in the file. That is the difference between a DATABASE and a FILE.

DEFINITION

The Table Vocabulary — Every Term You Need (1/2)

Column (also called Field or Attribute):
The vertical pillars of the table. They define the blueprint — WHAT information is stored. Each column has a NAME and a strict DATA TYPE. Example: first_name (text), salary (numeric), is_active (boolean). You cannot put 'twenty thousand' in a salary column — the database rejects it immediately.

Row (also called Record or Tuple):
The horizontal lines. Each row represents one complete real-world entity. One row = one customer OR one product OR one order. A table with 50,000 rows means 50,000 customers.

DEFINITION

The Table Vocabulary — Every Term You Need (2/2)

Schema:
A namespace (think: folder) that groups related tables inside a database. RetailMart V3 has schemas like 'customers', 'products', 'sales', 'finance'. Two different schemas can have tables with the same name — customers.addresses and stores.addresses can coexist without conflict.

Primary Key (PK):
A column that UNIQUELY identifies every row. Like Aadhaar for people — no two rows can EVER share the same PK value, and no row can have NULL as its PK. If you try to insert a duplicate? The database violently rejects it. Examples: customer_id, order_id, product_id.

Foreign Key (FK):
A column in one table that points to the Primary Key of ANOTHER table. This is the 'glue' that connects tables. Example: the orders table has cust_id (FK) pointing to customer_id (PK) in the customers table. You CANNOT place an order for a customer that does not exist — the FK prevents it. This is what makes a database 'relational'.

Two Connected Tables — Visualized (1/2)

TABLE: customers.customers — Schema: customers

customer_id (PK - INT)	first_name (VARCHAR)	last_name (VARCHAR)	email (VARCHAR)	
101	Rahul	Sharma	rahul.s@gmail.com	← ROW 1 (one customer)
102	Priya	Patel	priya.p@yahoo.com	← ROW 2
103	Amit	Kumar	amit.k@outlook.com	← ROW 3

customer_id = cust_id (FOREIGN KEY relationship)

TABLE: sales.orders — Schema: sales

order_id (PK)	cust_id (FK→customers)	store_id	order_date (DATE)	net_total (NUMERIC)	
5001	101	47	2025-03-15	2499.00	← Rahul's order
5002	103	12	2025-03-16	899.00	← Amit's order
5003	101	47	2025-04-01	5999.00	← Rahul again!

Two Connected Tables — Visualized (2/2)

```
└──────────┴──────────┴──────────┴──────────┴──────────┘


KEY OBSERVATIONS:
  • Rahul (101) has 2 orders  → One-to-Many relationship
  • Priya (102) has 0 orders  → She exists but never bought anything
  • cust_id 999 CANNOT exist in orders if no customer 999 exists → FK enforces this
```

DEFINITION

The Database Hierarchy — From Server to Cell (1/2)

```
[The Complete Stack]
PostgreSQL SERVER  (listens on port 5432)
  └── DATABASE: accio_retailmart_26  (one per batch)
        ├── SCHEMA: customers
        │     ├── TABLE: customers      (50,000 rows × 8 columns)
        │     ├── TABLE: addresses       (50,000 rows × 6 columns)
        │     ├── TABLE: reviews          (30,000 rows)
        │     ├── TABLE: loyalty_points
        │     └── TABLE: wallets
        ├── SCHEMA: sales
        │     ├── TABLE: orders           (150,000 rows × 9 columns)
        │     ├── TABLE: order_items      (374,753 rows)
        │     ├── TABLE: payments
        │     ├── TABLE: shipments
        │     └── TABLE: returns
        ├── SCHEMA: products
        │     ├── TABLE: products         (2,361 rows)
        │     ├── TABLE: suppliers
        │     ├── TABLE: inventory
```

DEFINITION

The Database Hierarchy — From Server to Cell (2/2)

```
│     └── TABLE: promotions
├── SCHEMA: finance
├── SCHEMA: hr
├── SCHEMA: marketing
├── SCHEMA: stores
├── SCHEMA: support
├── SCHEMA: web_events
├── SCHEMA: supply_chain
├── SCHEMA: loyalty
├── SCHEMA: manufacture
├── SCHEMA: payroll
├── SCHEMA: call_center
└── SCHEMA: audit
```

Server → Database → Schema → Table → Row → Cell (single value)

RetailMart V3: 16 schemas, 55 tables, ~2.7 million rows.
You will explore ALL of them by the capstone.

STORY

But Sir, I Can Do All This in Excel... (1/2)

Every batch, someone asks this. Let us settle it once and for all:

STORY

But Sir, I Can Do All This in Excel... (2/2)

[Excel vs PostgreSQL — The Honest Comparison]

Feature	Excel / Google Sheets	PostgreSQL
Max rows	1,048,576 (hard limit)	Billions (no practical limit)
Multiple users at once	Conflicts, overwriting	1000s of concurrent users
Data integrity	Anyone can type anything	Enforced types, constraints
Relationships	VLOOKUP (fragile)	Foreign Keys (guaranteed)
Speed on 1M rows	Freezes / crashes	< 1 second
Undo a mistake	Ctrl+Z (if you are lucky)	ROLLBACK (guaranteed)
Audit trail	None	Full change logging
Security	File-level password	Per-user, per-table permissions
Automation	Macros (limited)	Triggers, functions, schedules
Cost	₹5000/yr (Office 365)	₹0 (open source, forever)

Excel is a GREAT tool for quick analysis of small data.
PostgreSQL is for EVERYTHING ELSE.

Analogy: Excel is a bicycle. PostgreSQL is a train.
Both get you places. But only one handles 2.7 million passengers.

THINK ABOUT IT — Anatomy of a Table

Think like a database designer. Sketch your answers — we'll discuss together.

EASY

P2.1 — Name That Component

Look at this table and identify each component:

a) : How many COLUMNS does this table have? Name them.

b) : How many ROWS? What does each row represent?

c) : Which column is the PRIMARY KEY? How do you know?

d) : brand_id is a FOREIGN KEY. Which table and column does it point to?

e) : What data type would you assign to each column? (INT, VARCHAR, NUMERIC, DATE, BOOLEAN)

f) : Can two products have the same product_name? Can they have the same product_id? Why?

products.products (partial)

✅ ANSWER

product_id	product_name	price	brand_id
1	Samsung Galaxy S24	74999.00	5
2	Amul Butter 500g	299.00	12
3	Nike Air Max	8999.00	23

MEDIUM

P2.2 — Design a UPI Payment Database

PhonePe/Google Pay processes 10+ billion UPI transactions per month. You are hired to design their database:

Example to model: users(user_id [PK], name, phone, upi_id) — user_id uniquely identifies each person; other tables point back to it.

a) : List 4 tables you would create (e.g., users, transactions, ...). For each table, list 5-6 columns with data types.

b) : Mark the PRIMARY KEY for each table.

c) : Draw all FOREIGN KEY connections between your tables. Which table has the most FKs?

d) : Rahul sends ₹500 to Priya. Which tables get affected? List the exact columns that change.

e) : What happens if the server crashes AFTER debiting Rahul but BEFORE crediting Priya? Which database property prevents this disaster?

HARD

P2.3 — Spot the Design Mistakes

A junior developer designed this table. Find ALL the problems:

a) : There is no Primary Key. What problems will this cause? (Hint: there are two Rahuls)

b) : subjects and marks store comma-separated values. Why is this terrible? What breaks if you want average marks per subject?

c) : Redesign this using MULTIPLE tables with proper PKs, FKs, and data types. Draw the corrected schema.

d) : How many rows would your corrected design have for the same 3 students? Why is 'more rows' actually BETTER here?

Bad Table Design

✅ ANSWER

TABLE: student_data

name	phone	subjects	marks
Rahul	98765...	Math,Science,Eng	85,92,78
Priya	87654...	Math,Hindi	95,88
Rahul	76543...	Science	91

CRAZY

P2.4 — Reverse Engineer Swiggy

You open Swiggy. You search 'Biryani'. You see 47 restaurants with ratings, delivery times, prices, and offers. You place an order, pay via UPI, track the delivery, and leave a review.

Example to model: restaurants(restaurant_id [PK], name, area, rating) — one row per restaurant; the orders table references restaurant_id.

a) : List at least 8 TABLES Swiggy's database must have (restaurants, menu_items, orders, ...). For each table, list 5+ columns with data types.

b) : Draw the FOREIGN KEY relationships between all your tables. Which table is the 'center' of the design?

c) : Swiggy shows 'Delivered 2,847 orders from this restaurant.' Which tables and columns would you query to get this number?

d) : A delivery partner picks up the wrong order. Design a table that tracks delivery partner assignments and order status changes.

e) : Swiggy runs a '50% off up to ₹100' coupon. Design the 'coupons' table. Think about: expiry date, max uses per customer, minimum order value, applicable restaurants.

STORY

The English Accent Analogy

A student asked: 'Sir, I learned MySQL in college. Do I have to start over for PostgreSQL?'

Think of English. It is spoken in America, UK, Australia, and India. The core grammar is identical — subject, verb, object. They just use different slang:

```
[English 'Dialects']
American : 'I'll take the elevator to the apartment on the first floor.'
British  : 'I'll take the lift to the flat on the ground floor.'
Indian   : 'I'll take the lift to the flat on the ground floor, na?'

Same language. Different accent. You understand all three.
```

SQL works exactly the same way. About 90% of queries work identically across PostgreSQL, MySQL, Oracle, and SQL Server. The remaining 10% is each engine's 'accent'. Master one engine deeply, and switching to another takes days, not months.

We master PostgreSQL — the most feature-complete, open-source engine available. Everything you learn here works (with minor tweaks) on any SQL database you will ever encounter.

DEFINITION

SQL vs PostgreSQL — Language vs Engine

SQL (Structured Query Language):

A standardized LANGUAGE created in the 1970s for communicating with Relational Databases. Standardized by ANSI/ISO so the core commands work the same everywhere. SQL is the vocabulary and grammar.

PostgreSQL (The Engine / DBMS):

The actual SOFTWARE installed on your machine that understands SQL and physically executes your commands. It reads your query, plans the fastest route to the answer, touches the disk, grabs the data, and returns results.

```
[Think of it this way]
SQL        = The Hindi language (grammar + vocabulary)
PostgreSQL = A specific person who speaks Hindi fluently,
             has a photographic memory, can search through
             2.7 million records in milliseconds, and never
             forgets anything you tell them.


Other 'Hindi speakers': MySQL, Oracle, SQL Server, SQLite.
Same language. Different capabilities. Same core grammar.
```

DEFINITION

The 5 Categories of SQL Commands (1/2)

SQL has exactly 5 categories. These are asked in literally every data interview:

[SQL Command Categories — The Complete Map]

Category	Full Name	Purpose	Key Commands
DDL	Data Definition Language	Build/modify structures	CREATE, ALTER, DROP
DML	Data Manipulation Language	Insert/update/delete data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Read/retrieve data	SELECT
DCL	Data Control Language	Security & permissions	GRANT, REVOKE
TCL	Transaction Control Language	Save/undo changes	COMMIT, ROLLBACK

[The Building Analogy — Memorize This]

Imagine building a house:

DEFINITION

The 5 Categories of SQL Commands (2/2)

```
DDL  = The architect builds walls, rooms, doors     (CREATE TABLE, ALTER TABLE)
       If you demolish a room, that is DROP TABLE.

DML  = The movers bring furniture IN (INSERT),
       rearrange it (UPDATE), or throw it out (DELETE).

DQL  = You open the window and LOOK inside (SELECT).
       You do NOT move anything. Just observe.

DCL  = You decide who gets the house keys (GRANT)
       and take keys away from people (REVOKE).

TCL  = You decide: 'Keep this arrangement' (COMMIT)
       or 'Undo everything back to before' (ROLLBACK).
```

As a Data Analyst, 90%+ of your work will be DQL (SELECT). You describe WHAT data you want, and PostgreSQL figures out HOW to get it. That is what 'declarative' means.

STORY

Declarative vs Imperative — The Restaurant Analogy

You walk into a restaurant. Two ways to get food:

IMPERATIVE (Step-by-step instructions):

'Go to the kitchen. Open the second fridge. Take out the paneer from the middle shelf. Wash it. Cut it into cubes. Heat oil in a kadhai to 180°C. Add cumin seeds. Wait 10 seconds. Add onions...'

You told the chef HOW to cook, step by step. If you miss one step — disaster.

DECLARATIVE (Just say what you want):

'One paneer butter masala, medium spice.'

You told the chef WHAT you want. The chef decides HOW — which fridge, which pan, what temperature. The chef is the expert. You trust the chef.

SQL is DECLARATIVE.

You say: 'Give me all customers from Mumbai who spent more than ₹10,000.' PostgreSQL's Query Planner (the chef) decides: Should I scan every row? Use an index? Which join algorithm? You do not care. You just want the answer.

MEDIUM

Scenario: Imperative (Python) vs Declarative (SQL) — Same Problem

Task: Find all customers from Mumbai.

SQL tells WHAT you want. The PostgreSQL engine's Query Planner automatically decides HOW — index scan, parallel workers, memory allocation. That is why SQL has outlived 5 generations of programming languages.

Python — Imperative (You tell HOW)

✅ ANSWER

```
# Open the file
customers = open('customers.csv')
results = []

# Loop through every single row — one by one
for line in customers:
    fields = line.split(',')
    if fields[5] == 'Mumbai':        # Check city column
        results.append(fields)

for r in results:
    print(r)

# Lines of code: 8
# Time on 50,000 rows: ~2 seconds
# Time on 5,000,000 rows: ~200 seconds (3+ minutes)
```

SQL — Declarative (You tell WHAT)

✅ ANSWER

```
SELECT * FROM customers.customers WHERE tier = 'Gold';

-- Lines of code: 1
-- Time on 50,000 rows: ~3 milliseconds
-- Time on 5,000,000 rows: ~50 milliseconds (with index)
-- That is 4000x faster. And 8x less code.
```

HARD

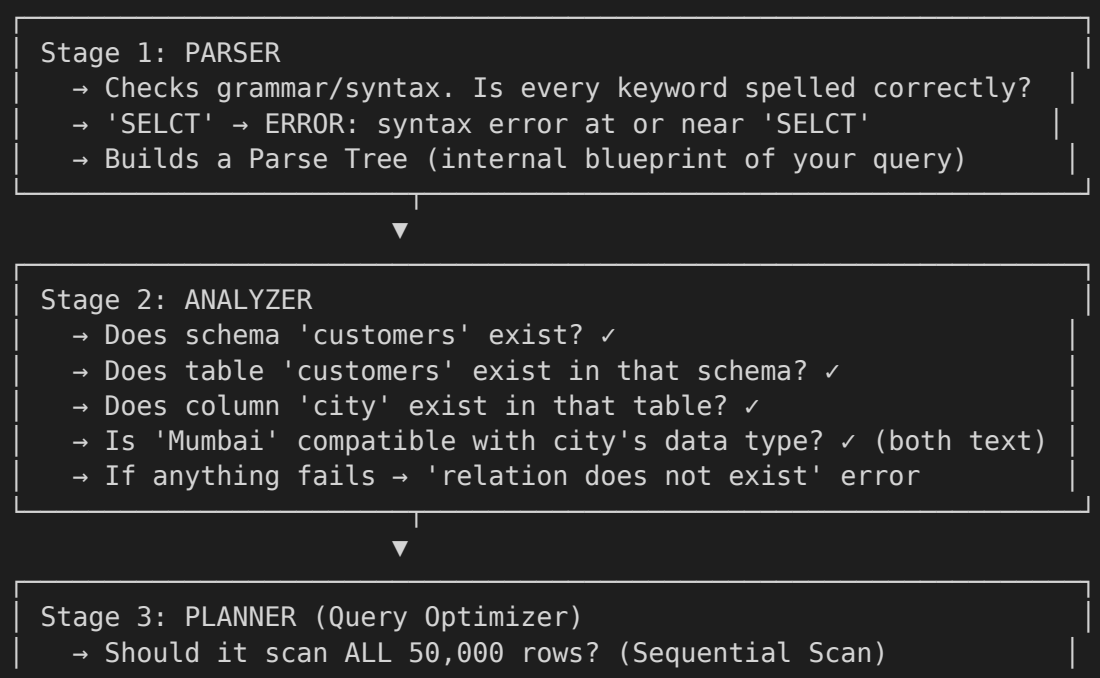
Scenario: What Happens When You Press Execute — The 4-Stage Pipeline
Every SQL query goes through 4 stages inside PostgreSQL. Understanding this will save you hours of debugging later:

On Query Plans (Reading Query Plans), you will learn to read the Planner's internal statistics using EXPLAIN ANALYZE. For now, just know: every query passes through Parser → Analyzer → Planner → Executor, and the Planner is what makes SQL fast without you doing anything.

The SQL Query Pipeline (1/2)

✅ ANSWER

YOU TYPE: SELECT * FROM customers.customers WHERE city = 'Mumbai';
YOU PRESS: F5 (Execute) in pgAdmin



The SQL Query Pipeline (2/2)

✅ ANSWER

```
|   → Or is there an index on 'city'? (Index Scan — much faster)  |
|   → It picks the CHEAPEST execution route automatically.        |
|   → This is the magic — YOU do not decide. The engine does.     |
└─────────────────────────┬──────────────────────────────────────┘
                          ▼
┌──────────────────────────────────────────────────────────────┐
| Stage 4: EXECUTOR                                            |
|   → Touches the disk, grabs the matching rows                |
|   → Returns the result set to your screen (pgAdmin / psql)   |
|   → Total time: ~3 milliseconds for 50,000 rows              |
└──────────────────────────────────────────────────────────────┘
```

THINK ABOUT IT — SQL: The Language

Reason through these together — it's about understanding, not memorising.

EASY

P3.1 — Classify the Commands

Put each SQL command into its category (DDL, DML, DQL, DCL, or TCL):

a) : Group all 12 commands by category.

b) : Which category appears most often in this list?

c) : Which category will a Data Analyst use 90% of the time? Why?

d) : A Data Engineer runs DELETE, INSERT, CREATE TABLE daily. Which categories does a DE use most?

Classify These

✅ ANSWER

1. `CREATE TABLE students (id INT, name VARCHAR(100));`
2. `SELECT * FROM orders WHERE amount > 500;`
3. `INSERT INTO products (name, price) VALUES ('Laptop', 45000);`
4. `GRANT SELECT ON customers TO analyst_user;`
5. `DROP DATABASE test_db;`
6. `UPDATE employees SET salary = salary * 1.10;`
7. `COMMIT;`
8. `ALTER TABLE orders ADD COLUMN discount NUMERIC;`
9. `DELETE FROM logs WHERE created_at < '2024-01-01';`
10. `ROLLBACK;`
11. `REVOKE INSERT ON products FROM junior_dev;`
12. `SELECT COUNT(*) FROM sales.orders GROUP BY store_id;`

MEDIUM

P3.2 — Pipeline Detective

For each query below, identify which stage of the Query Pipeline (Parser, Analyzer, Planner, Executor) catches the problem. Explain WHY that stage:

a) : SELECT * FORM customers; (look carefully at the keyword)

b) : SELECT firstname FROM customers.customers; (the actual column name is first_name with an underscore)

c) : SELECT * FROM customers.customers WHERE city = 'Mumbai'; (no error — but should it scan all 50,000 rows or use an index?)

d) : SELECT * FROM secret_schema.secret_table; (this table does not exist)

e) : SELECT * FROM sales.orders WHERE order_date > 'banana'; (comparing a date column to a fruit)

Bonus: Which stage is the ONLY one that actually touches the data on disk?

HARD

P3.3 — Imperative to Declarative Translation

Below are step-by-step IMPERATIVE descriptions. Convert each to a single DECLARATIVE SQL-like statement (do not worry about exact syntax — just express the intent):

a) : Open the employees file. Loop through every row. Check if salary > 100000. If yes, print that row. → Write this as a SELECT … WHERE …

b) : Open the orders file. For each row, check if order_date is in 2025. If yes, add the net_total to a running sum. Print the sum. → Write as SELECT SUM(…) … WHERE …

c) : Open the products file. Group all products by category. For each group, count how many products exist. Print category name and count. → Write as SELECT … GROUP BY …

d) : For each customer, open the orders file and check if they have any orders. If a customer has zero orders, print their name. → Think: which TWO tables do you need?

You will write REAL SQL for these starting Filtering & Sorting. For now, just practice thinking declaratively — WHAT, not HOW.

CRAZY

P3.4 — Why Did SQL Survive 51 Years?

This is an essay-style question. Think deeply:

Programming languages come and go. COBOL (1959), Pascal (1970), Perl (1987), Flash/ActionScript (2000) — all popular in their time, all fading or dead.

SQL was created in 1974. It is 51 years old. Yet it is the #1 skill in data job postings in 2025. More popular today than ever.

a) : Give 3 technical reasons SQL survived (think: declarative nature, standardization, what problem it solves).
b) : Give 2 business reasons SQL survived (think: who uses it, what decisions it powers, cost of switching).
c) : Google created Spanner (2012) — a globally distributed database that speaks SQL. Facebook created Presto (2012) — a query engine for big data that speaks SQL. What does this tell you about SQL's future?
d) : A startup CEO says: 'We do not need SQL. We will use AI to query our data with natural language.' Write a 3-sentence counter-argument.

STORY

A Day in the Life of a Data Analyst at Flipkart (1/2)

Meet Sneha. She joined Flipkart as a Data Analyst 6 months ago. Here is her Tuesday:

```
[Sneha's Day — Every Task is SQL]
9:00 AM   Check the daily dashboard
          → She wrote the SQL queries behind each chart.
          → 'Yesterday's GMV was ₹47 Cr. Down 3% from Monday.'


9:30 AM   Slack message from the VP of Sales:
          'Top 10 sellers in Electronics with >50% return rate?'
          → Sneha writes a JOIN across 3 tables, adds GROUP BY,
            filters with HAVING. 15 minutes. Done.
```

STORY

A Day in the Life of a Data Analyst at Flipkart (2/2)

```
11:00 AM  Marketing wants to know: 'Which customers bought in
          January but NOT in February? Send them a coupon.'
          → Sneha writes a subquery with NOT EXISTS.
          → Exports 23,847 customer emails to the marketing tool.

2:00 PM   Finance asks: 'Monthly revenue trend, last 12 months,
          broken down by payment method.'
          → Sneha uses a Window Function with SUM() OVER().
          → Creates a materialized view so Finance can self-serve.

4:00 PM   Sneha notices something odd in the data — 847 orders
          have order_date BEFORE the customer's registration_date.
          → She flags it to Engineering with the exact query.
          → Data quality issue fixed the same day.

5:30 PM   Logs off. Salary: ₹12 LPA. 8 months of experience.
          Tool used all day: PostgreSQL + pgAdmin. That is it.
```

Every single task Sneha did today, you will learn in this course. The JOIN query? Joins Part 1. The subquery? Subqueries Part 2. The window function? Window Functions Part 1. The data quality check? Data Quality.

DEFINITION

The Data Professional's Toolkit

[The Learning Path — Where SQL Fits]

Stage	Tool / Skill	What It Does	When You Learn It
1	Excel / Sheets	Quick analysis, small datasets	Already know it
2	SQL	Query databases, transform data	THIS COURSE (6 wks)
3	Python	Automation, ML, scripting	After SQL
4	Power BI/Tableau	Dashboards, visual storytelling	After SQL + Python

Why SQL FIRST? Three reasons:

1. 80% of all data lives in databases. Cannot analyze what you cannot access.
2. Power BI and Python BOTH connect to databases via SQL.
3. Every data interview starts with 'Write a SQL query to...'

SQL is not one of many skills. It is THE foundational skill.
Everything else plugs into it.

SQL Career Map — Who Uses SQL and How Much

Role	SQL Usage	What They Do With SQL	Avg Salary (India)
Data Analyst	80% of work	Reports, dashboards, ad-hoc queries	₹6-15 LPA
Business Intelligence	60%	KPI tracking, exec reporting	₹8-18 LPA
Data Scientist	40%	Pull training data, feature engineering	₹10-25 LPA
Data Engineer	70%	ETL pipelines, data transformations	₹12-30 LPA
Backend Developer	40%	CRUD operations, query optimization	₹8-25 LPA
Database Administrator	90%	Performance tuning, backups, security	₹10-22 LPA

There is NO data career where SQL is optional.

SQL vs Python — They Are Partners, Not Competitors

Task	SQL	Python
Filter 1M rows by date range	50ms (native)	3s (pandas)
JOIN 3 tables on keys	Optimized by engine	Manual pd.merge()
GROUP BY + aggregation	One-liner	df.groupby()
Build a ML model	Not possible	scikit-learn
Create a web dashboard	Not possible	Streamlit/Dash
Visualize a chart	Not possible	Matplotlib/Plotly

The Rule: SQL EXTRACTS and TRANSFORMS data (fast, inside the database).
Python ANALYZES and VISUALIZES (flexible, outside the database).

In practice: Sneha writes SQL to pull 50,000 rows → exports to Python
→ builds a churn prediction model → presents results in Power BI.
SQL starts the pipeline. Python finishes it.

REAL WORLD

REAL WORLD

COMPANIES POWERED BY POSTGRESQL

Every time you use these apps, PostgreSQL queries run behind the scenes:

[PostgreSQL in the Wild]

Company	What PostgreSQL Handles	Scale
Instagram	Photo metadata, user profiles, likes	500M+ daily users
Spotify	Playlists, user libraries, play history	100B+ plays logged
Apple iCloud	Account data, device sync	1B+ devices
Uber	Trip data, driver locations, payments	25M+ trips/day
IRCTC	Railway reservations, PNR status	2M+ bookings/day
Razorpay	Payment processing, merchant data	Handles ₹lakh crores
Zerodha	Trading records, portfolio data	10M+ accounts

When you write SELECT * FROM orders in pgAdmin, you are using THE SAME engine running at Instagram and Razorpay. Same commands, same skills — just scaled differently.

THINK ABOUT IT — Roles & Career Paths

Think about YOUR path — be honest, there are no wrong answers.

EASY

P4.1 — Match the Role to the Task

Match each task to the most likely role (Data Analyst, Data Engineer, Data Scientist, DBA, Backend Developer):

a) : Build a dashboard showing monthly revenue trends for the CEO

b) : Set up an automated pipeline that loads CSV files into PostgreSQL every night at 2 AM

c) : Train a machine learning model to predict which customers will churn

d) : Optimize a slow query that takes 45 seconds to run — make it run in 200ms

e) : Write an API endpoint that returns a customer's order history when they log into the app

f) : Investigate why 847 orders have dates before the customer's registration — find and fix the data quality issue

MEDIUM

P4.2 — SQL or Python? Choose the Right Tool

For each task, decide whether SQL or Python is the better tool. Explain why:

a) : Find the top 10 products by revenue across 150,000 orders

b) : Create a bar chart showing monthly sales for the last 12 months

c) : Calculate the running total of revenue, month over month

d) : Send an automated email alert when daily revenue drops below ₹5 lakhs

e) : Join 4 tables to find customers who bought Electronics but never Fashion

f) : Build a recommendation engine: 'Customers who bought X also bought Y'

HARD

P4.3 — Build Sneha's Query

Go back to Sneha's day (the Flipkart analyst story). For each task she did, identify:

a) : 9:30 AM task — 'Top 10 sellers with >50% return rate': Which 3 tables does she need? What columns? What SQL concepts (JOIN, GROUP BY, HAVING)?

b) : 11:00 AM task — 'Bought in Jan but NOT Feb': Which tables? What is the concept called when you find rows that do NOT match? (Hint: earlier sessions)

c) : 2:00 PM task — 'Monthly revenue by payment method': Which tables? Why did she use a 'materialized view' instead of just giving Finance the query?

d) : 4:00 PM task — 'Orders before registration': Write the LOGIC (not SQL) for finding these bad records. Which two columns do you compare?

CRAZY

P4.4 — Your Personal Career Map

This is about YOU. Answer honestly:

a) : Which role interests you most? Data Analyst, Data Scientist, Data Engineer, or something else? Why?

b) : List 3 companies you would love to work at. For each, guess what their database probably stores and how SQL would be used there.

c) : By the capstone of this course, you will be able to write queries across 55 tables with 2.7 million rows. How would you describe this skill on your resume in ONE bullet point?

d) : Write a 2-sentence LinkedIn post about starting your SQL journey today. Make it professional, specific, and genuine.

STORY

You Will Never See Apps the Same Way Again

After today, every app on your phone will look different. You will not see the pretty UI anymore. You will see the DATABASE behind it. This is your 'Matrix moment' — once you see the tables, you cannot unsee them.

[Decode Every App On Your Phone]

APP	What YOU See	What the DATABASE Sees
Instagram	A photo with 247 likes	INSERT INTO posts; 247 rows in likes table
Swiggy	'Delivered in 28 mins'	UPDATE orders SET status='Delivered'
PhonePe	'₹500 sent to Rahul'	BEGIN; UPDATE accounts; UPDATE accounts; COMMIT;
Ola/Uber	Driver 3 mins away	SELECT * FROM drivers WHERE distance < 3km
Netflix	'Because you watched...'	SELECT * FROM shows WHERE genre IN (your history)
IRCTC	'Seat 23, Coach S4'	UPDATE seats SET booked=true WHERE seat_id=23
Myntra	'70% off!' banner	SELECT * FROM promotions WHERE active=true
Cred	Credit score: 784	SELECT credit_score FROM users WHERE id = you

Every swipe, tap, search, and payment = a SQL query somewhere.
You are about to learn the language these apps speak.

INTERVIEW QUESTION BANK — SQL Intro Concepts

These are ACTUAL questions from Data Analyst and Data Engineer interviews at Indian companies. Practice answering out loud — not in your head, OUT LOUD. Interviews test articulation, not just knowledge.

INTERVIEW

Interview Question Bank

Q1: 'What is a database and why can we not just use files?'

A: 'A database is an organized collection of data managed by a DBMS. Files lack data type enforcement, concurrent write safety, indexing, transactions, and access control. A database provides all of these — which is why we trust it with payments and medical records, not a CSV file.'

Q2: 'What is the difference between SQL and NoSQL?'

A: 'SQL databases store data in structured tables with predefined schemas and enforce relationships via Foreign Keys. They guarantee ACID — ideal for banking, e-commerce, analytics. NoSQL databases use flexible formats (JSON, key-value, graph). They excel at horizontal scaling and unstructured data — ideal for caching, chat, IoT. Most production systems use both: PostgreSQL for core data, Redis/MongoDB for caching.'

Q3: 'What is a Primary Key?'

A: 'A column that uniquely identifies every row. Cannot be NULL, cannot have duplicates. Creates an internal index for fast lookups. Other tables reference it via Foreign Keys.'

Q4: 'What is a Foreign Key?'

REVEAL

SQL Intro Self-Check (12 Questions)

Close your notes. Answer ALL of these from memory:

- 1. What is the difference between Data and Information? Give a real example.
- 2. What is a DBMS? What is an RDBMS? Is PostgreSQL a DBMS or RDBMS?
- 3. Name 3 scenarios where SQL is better than NoSQL. Name 2 where NoSQL wins.
- 4. What are the 5 SQL sub-languages? Give one command from each.
- 5. What is a Primary Key? Why can two rows NOT share the same PK?
- 6. What is a Foreign Key? What problem does it solve?
- 7. What is the database hierarchy? (Server → ??? → ??? → ??? → ??? → ???)
- 8. Is SQL declarative or imperative? Explain the difference in one sentence.
- 9. What are the 4 stages of the Query Pipeline? Which stage decides HOW to execute?
- 10. Name 3 careers that use SQL daily. What does each do with it?
- 11. Excel can handle 1 million rows. Why do we still need PostgreSQL?
- 12. Name 3 Indian companies that use PostgreSQL. What do they store in it?

Score: 10+ correct → ready for Setup & Onboarding. Below 10 → re-read the weak sections.

RECAP

Key Takeaways (1/2)

1. Data → Information → Database: Raw data is meaningless. Add structure and context → information. A Database stores it. A DBMS (PostgreSQL) manages, protects, and queries it.

2. SQL vs NoSQL: SQL = strict tables, ACID guarantees, relationships via keys. NoSQL = flexible schemas, horizontal scaling, eventual consistency. 80% of enterprise data is in SQL. Most companies use both.

3. The Table: Columns = blueprint with enforced data types. Rows = individual records. Primary Key = unique identifier. Foreign Key = relationship glue between tables.

RECAP

Key Takeaways (2/2)

4. The Hierarchy: Server → Database → Schema → Table → Row → Cell. RetailMart V3 has 16 schemas, 55 tables, ~2.7M rows.

5. SQL Categories: DDL (structures), DML (data changes), DQL (reads — 90% of analyst work), DCL (permissions), TCL (transactions).

6. Declarative Power: Tell SQL WHAT you want. The Query Pipeline (Parse → Analyze → Plan → Execute) figures out HOW. This is why SQL outlived 51 years of technology changes.

7. Career Foundation: Excel → SQL → Python → Power BI. SQL is mandatory for every data career. Master PostgreSQL, switching engines takes days.

SQL Intro — Quick Reference Card (Keep This!)

-- SQL Categories

```
DDL  — CREATE, ALTER, DROP       (structures)
DML  — INSERT, UPDATE, DELETE     (data changes)
DQL  — SELECT                     (reading data — 90% of analyst work)
DCL  — GRANT, REVOKE             (permissions)
TCL  — COMMIT, ROLLBACK          (transactions)
```

-- Database Hierarchy

```
Server → Database → Schema → Table → Row → Cell
PostgreSQL Server listens on Port 5432
Schema = namespace (folder for related tables)
RetailMart V3: 16 schemas, 55 tables, ~2.7M rows
```

-- Key Terms

```
RDBMS       = Relational Database Management System
Primary Key = Unique identifier per row (never NULL, never duplicate)
Foreign Key = Link between two tables (prevents orphan records)
ACID        = Atomicity, Consistency, Isolation, Durability
Declarative = Tell WHAT you want, not HOW to get it
```

SQL Intro — Quick Reference Card (Keep This!)

-- Query Pipeline (Every Query Passes Through This)

1. Parser   — Check grammar / syntax (catches typos)
2. Analyzer — Check tables/columns exist (catches wrong names)
3. Planner   — Choose cheapest execution path (the 'brain')
4. Executor — Touch disk, return results (the 'hands')

-- SQL vs NoSQL — When to Use What

SQL   → Banking, e-commerce, analytics, anything needing ACID
NoSQL → Chat, caching, IoT, logs, flexible/unstructured data
Real world: most companies use BOTH (Polyglot Persistence)

CHALLENGE

CHALLENGE

SQL Intro Take-Home Challenge

Complete these tasks BEFORE Setup & Onboarding:

1. Download (do NOT install yet): : PostgreSQL 18, pgAdmin 4, VS Code, Git. We install together on Setup & Onboarding.
2. Review: : Re-read today's notes. Write down 3 things that surprised you.
3. App Reverse Engineering: : Pick one app you use daily (Swiggy, Ola, PhonePe, etc.). List 6 tables its database probably has, with 5 columns each (include data types and mark PKs/FKs).
4. The 51-Year Question: : Research one more reason why SQL survived 51 years. Write it down in your own words.
5. Self-Check: : Go through the 12 self-check questions again. If you score below 10, re-read the weak sections.

Course Portal: https://starlordali4444.github.io/PostgreSql/

We hit the ground running — installation, first connection, and exploring RetailMart V3's 55 tables!

YOUR SQL JOURNEY

1. **SQL Intro ←YOU ARE HERE** ← HERE
2. Installation
3. DDL + Types
4. Constraints + DML
5. Filtering (first queries!)
9. Joins
13. CTEs
16. Window Functions
20. Indexing
25. Views
27. Cohort/RFM
29. Capstone

📱 WhatsApp Announcement

Tomorrow we go from concepts to HANDS-ON:

- Install PostgreSQL 18 + pgAdmin 4 + VS Code + Git together in class
- Understand the Client-Server architecture
- Connect via pgAdmin and psql
- Import RetailMart V3 — 16 schemas, 55 tables, 2.7 million rows of real data
- Tour the database: list schemas, count tables, explore sales.orders
- Write your FIRST SQL queries: SELECT count(*), SELECT * LIMIT 5

Come with PostgreSQL and VS Code DOWNLOADED (not installed — we do that together).

— See you on Setup & Onboarding! Siraj Sir